Sets in Go: past, present and future

Luciano Ramalho
luciano@ramalho.org
github/codeberg:
@ramalho

My best work (so far): Fluent Python

- Published in 9 languages, 2 editions (2015, 2022)
- A deep dive into idiomatic Python, exploring the design of the language and its standard library

The Typing Map

The four typing approaches depicted in Figure 13-1 are complementary: they have different pros and cons. It doesn't make sense to dismiss any of them.

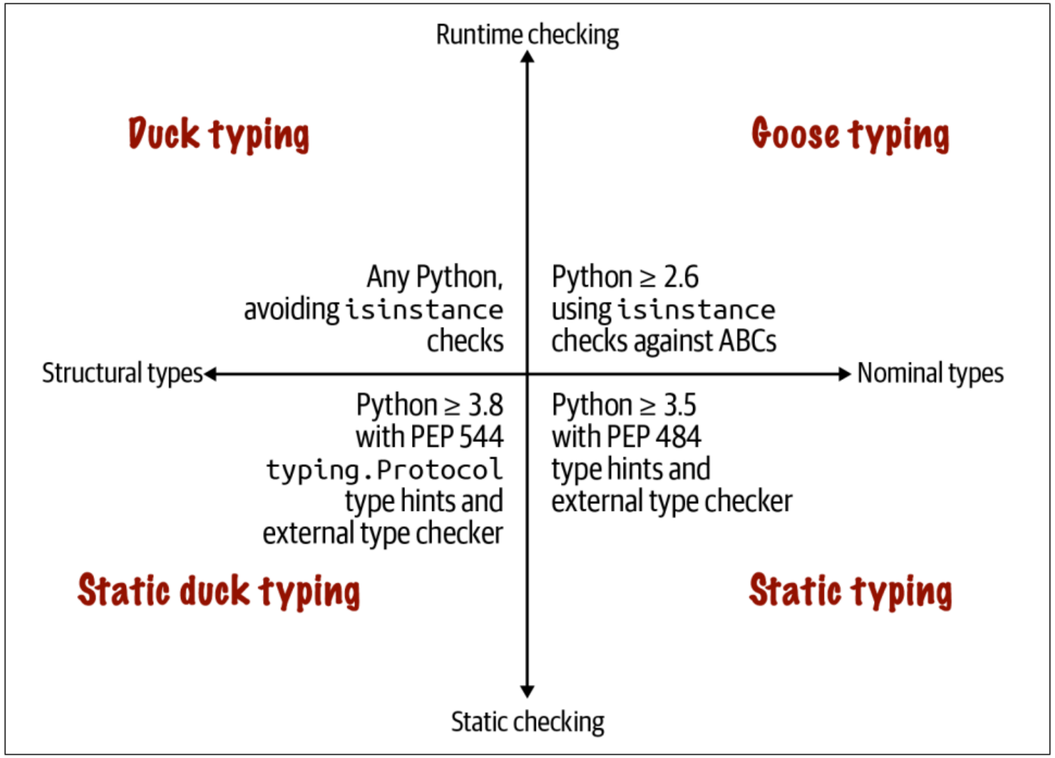


Figure 13-1. The top half describes runtime type checking approaches using just the Python interpreter; the bottom requires an external static type checker such as MyPy or an IDE like PyCharm. The left quadrants cover typing based on the object's structure—i.e., the methods provided by the object, regardless of the name of its class or superclasses; the right quadrants depend on objects having explicitly named types: the name of the object's class, or the name of its superclasses.

Typing Approaches in Popular Languages

Figure 13-8 is a variation of the Typing Map (Figure 13-1) with the names of a few popular languages that support each of the typing approaches.

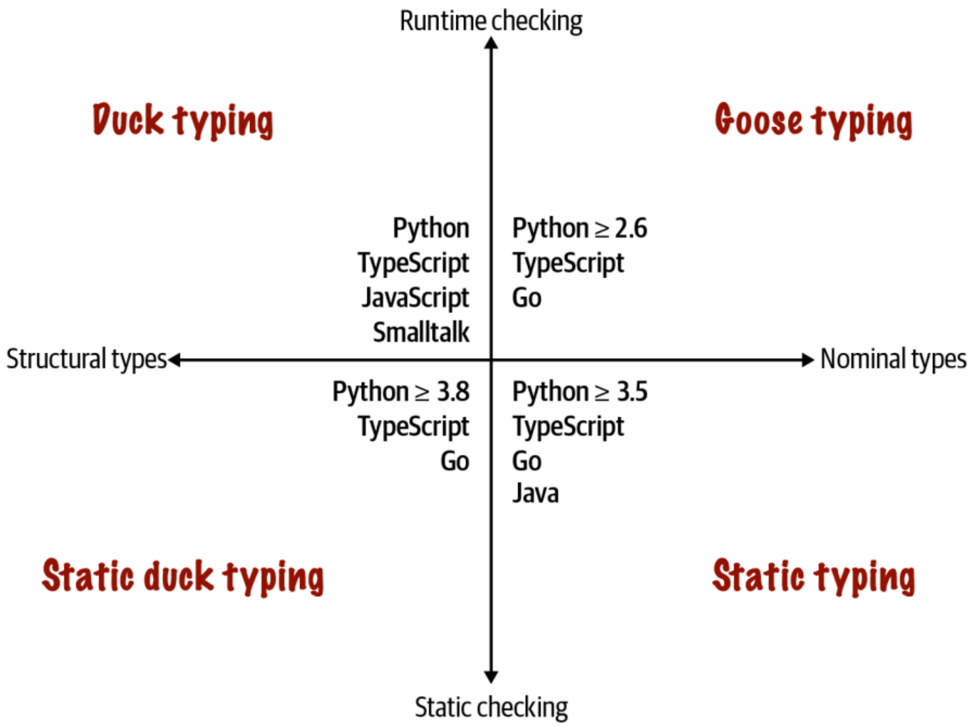


Figure 13-8. Four approaches to type checking and some languages that support them.

TypeScript and Python ≥ 3.8 are the only languages in my small and arbitrary sample that support all four approaches.

Go is clearly a statically typed language in the Pascal tradition, but it pioneered static duck typing—at least among languages that are widely used today. I also put Go in the goose typing quadrant because of its type assertions, which allow checking and adapting to different types at runtime.

Typing Approaches in Popular Languages

Figure 13-8 is a variation of the Typing Map (Figure 13-1) with the names of a few popular languages that support each of the typing approaches.

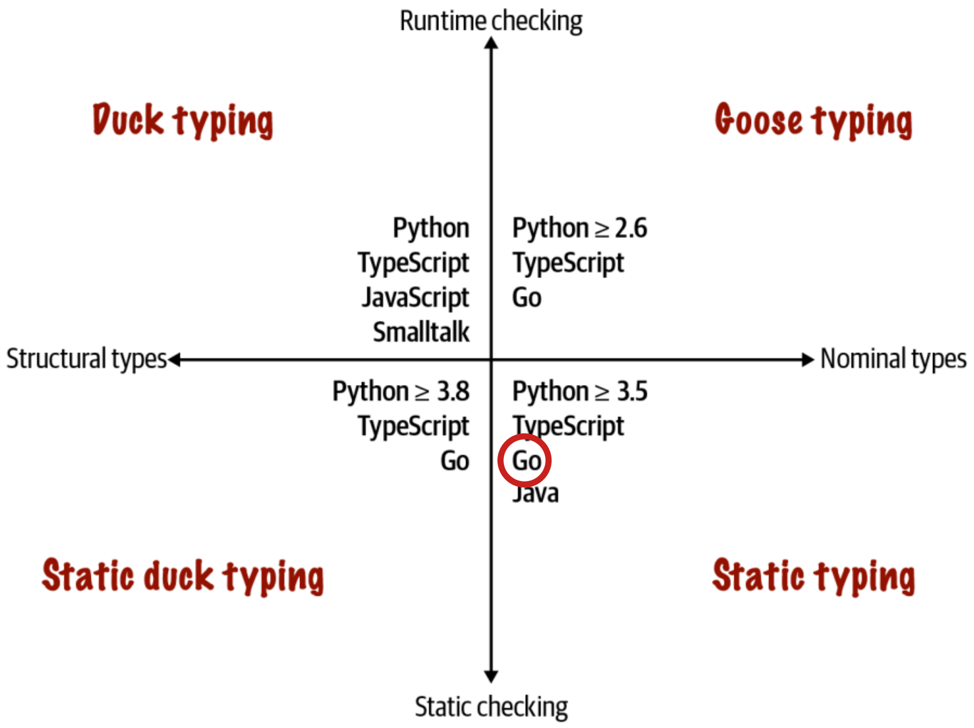


Figure 13-8. Four approaches to type checking and some languages that support them.

TypeScript and Python ≥ 3.8 are the only languages in my small and arbitrary sample that support all four approaches.

Go is clearly a statically typed language in the Pascal tradition, but it pioneered static duck typing—at least among languages that are widely used today. I also put Go in the goose typing quadrant because of its type assertions, which allow checking and adapting to different types at runtime.

Typing Approaches in Popular Languages

Figure 13-8 is a variation of the Typing Map (Figure 13-1) with the names of a few popular languages that support each of the typing approaches.

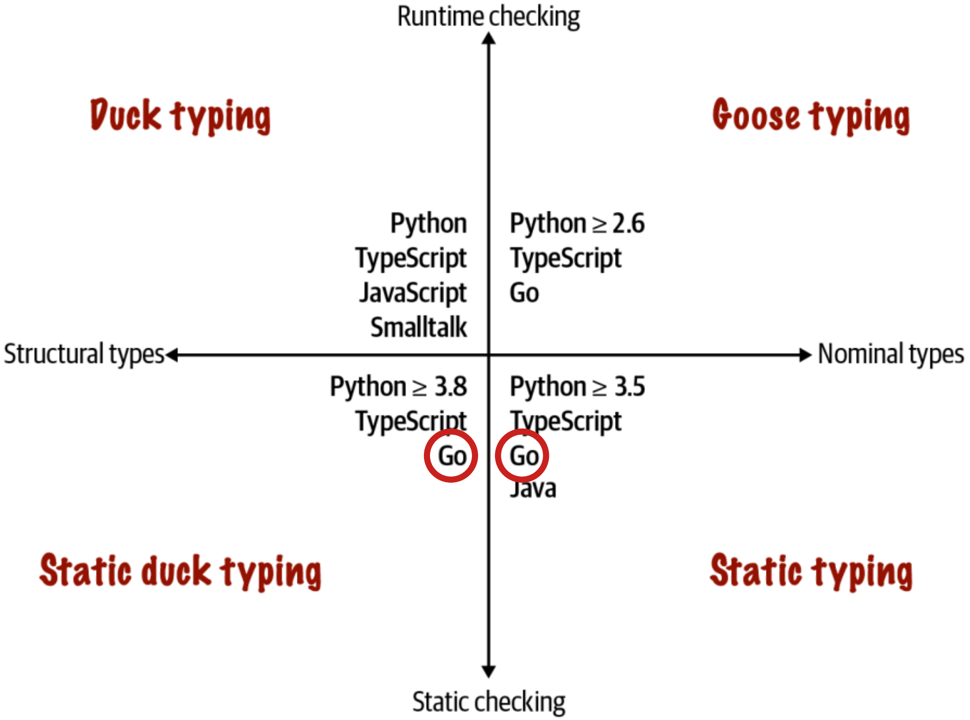


Figure 13-8. Four approaches to type checking and some languages that support them.

TypeScript and Python ≥ 3.8 are the only languages in my small and arbitrary sample that support all four approaches.

Go is clearly a statically typed language in the Pascal tradition, but it pioneered static duck typing—at least among languages that are widely used today. I also put Go in the goose typing quadrant because of its type assertions, which allow checking and adapting to different types at runtime.

Typing Approaches in Popular Languages

Figure 13-8 is a variation of the Typing Map (Figure 13-1) with the names of a few popular languages that support each of the typing approaches.

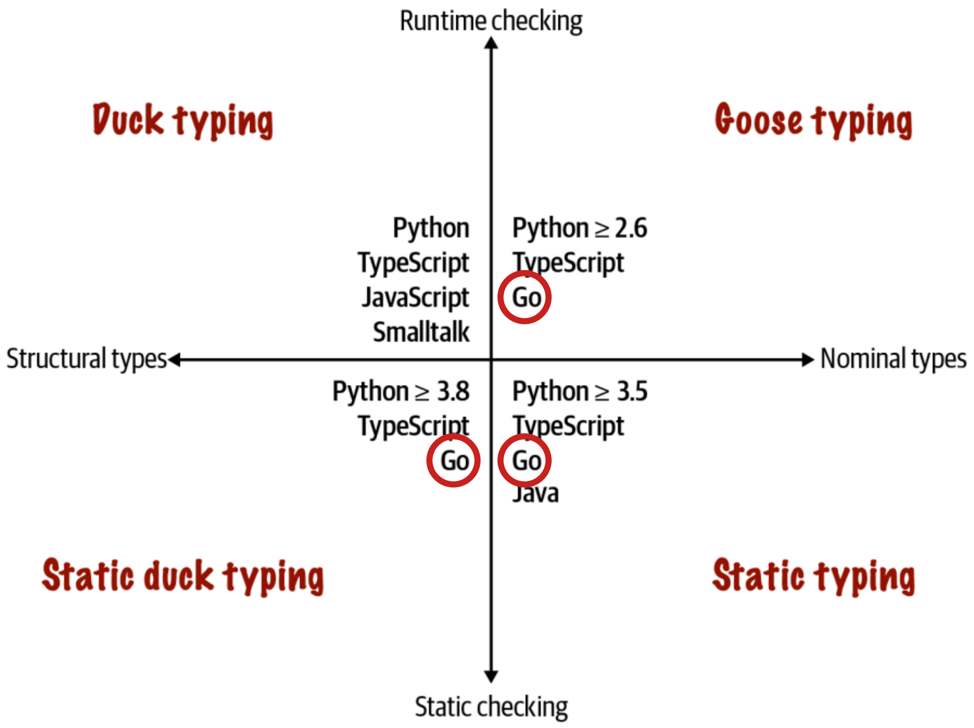


Figure 13-8. Four approaches to type checking and some languages that support them.

TypeScript and Python ≥ 3.8 are the only languages in my small and arbitrary sample that support all four approaches.

Go is clearly a statically typed language in the Pascal tradition, but it pioneered static duck typing—at least among languages that are widely used today. I also put Go in the goose typing quadrant because of its type assertions, which allow checking and adapting to different types at runtime.

About me

- Member of the Bartz v. Anthropic class action
- Principal consultant at Thoughtworks (2015-2023)
- Instructional designer at Oficina Turing
- Co-founder of Garoa Hacker Clube,
  a hackerspace in São Paulo, Brasil (since 2010)

A tale of two talks

- In 2018 I presented "Set Practice" at GopherCon Brasil:
  - Pitch: "Why and how to implement sets in Go"
- This is a radical update, covering the set types in the collections package for Go 1.28

USE CASE #1

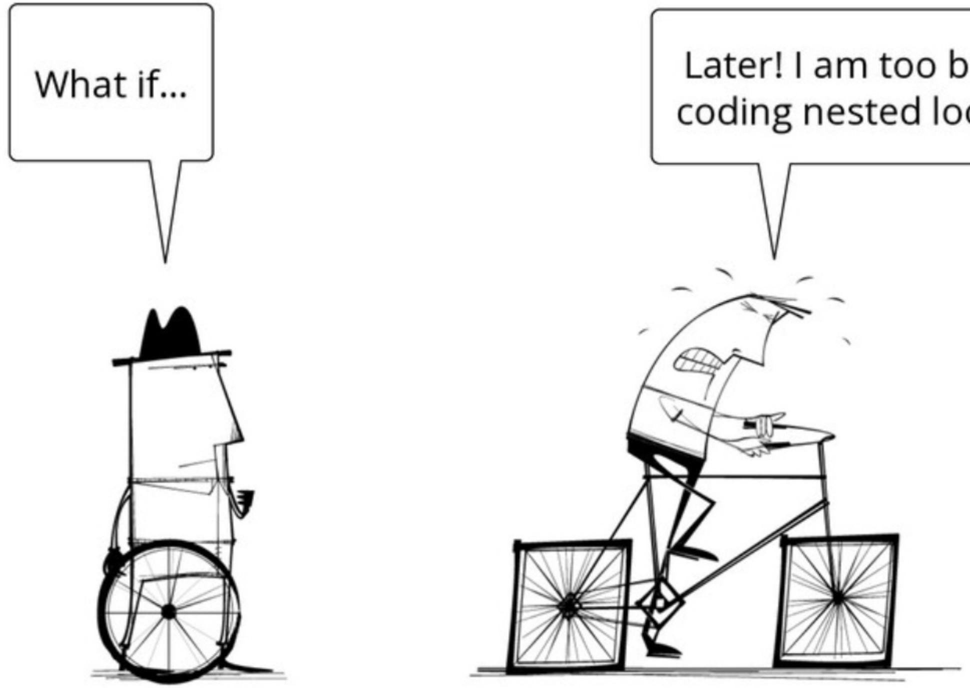
display product if all words in the query appear in the product description.

HAND-ROLLED SOLUTION #1

vintage 2018

I've written code like this in Go, which lacks built-in sets:

```
func ContainsAll(slice, subslice []string) bool {
    for _, needle := range subslice {
        found := false
        for _, elem := range slice {
            if needle == elem {
                found = true
                break
            }
        }
        if !found {
            return false
        }
    }
    return true
}
```

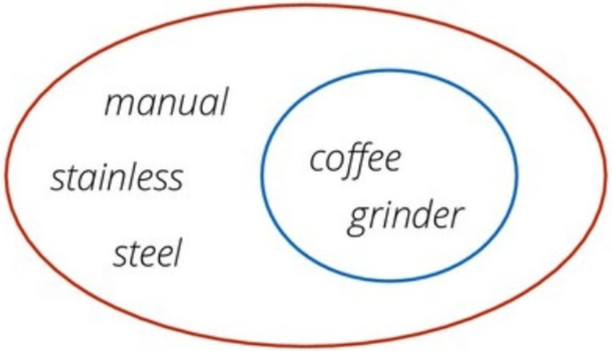


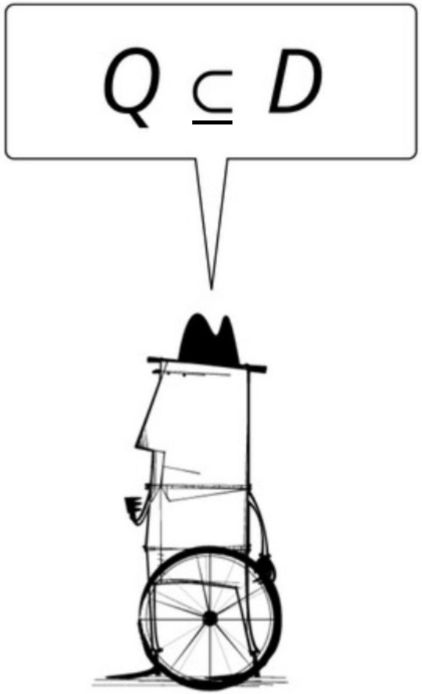
www.workcompass.com/

USE CASE #1

www.workcompass.com/

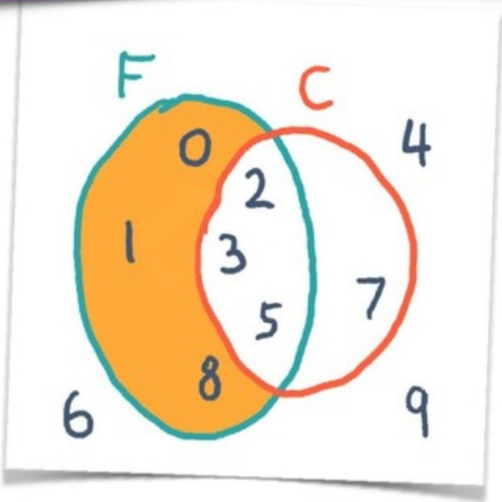
display product if all words in the query appear in the product description.

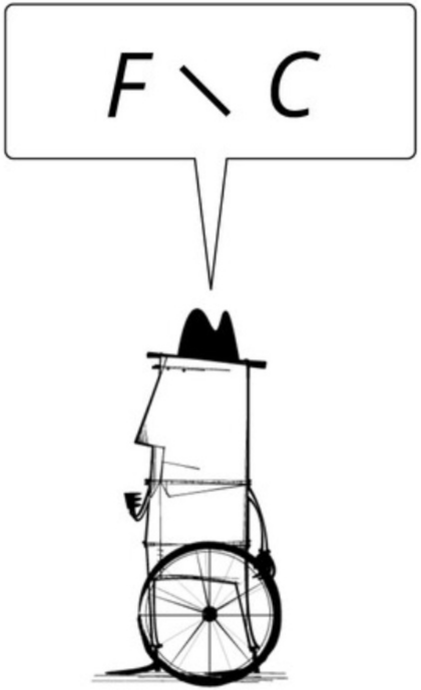




USE CASE #2

Mark all products previously favorited, except those already in the shopping cart.





Sets and Logic

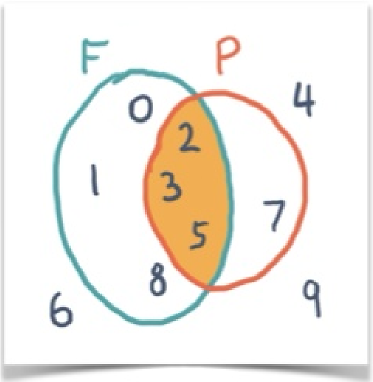
Nobody has yet discovered a branch of mathematics that has successfully resisted formalization into set theory.

Thomas Forster

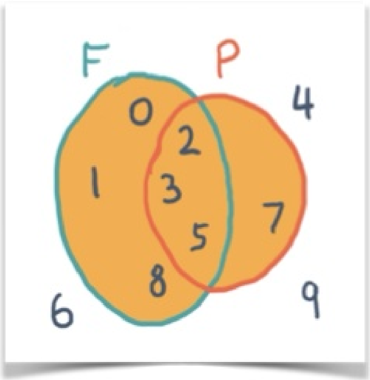
Logic Induction and Sets

p. 167

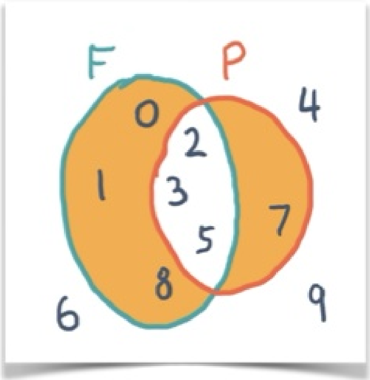
SOME SET OPERATIONS AND LOGIC EQUIVALENTS



Intersection
a.k.a. logical conjunction



Union
a.k.a. logical disjunction



Symmetric difference
a.k.a. negation of conjunction

LOGIC CONJUNCTION IS INTERSECTION

x belongs to the intersection of A with B.

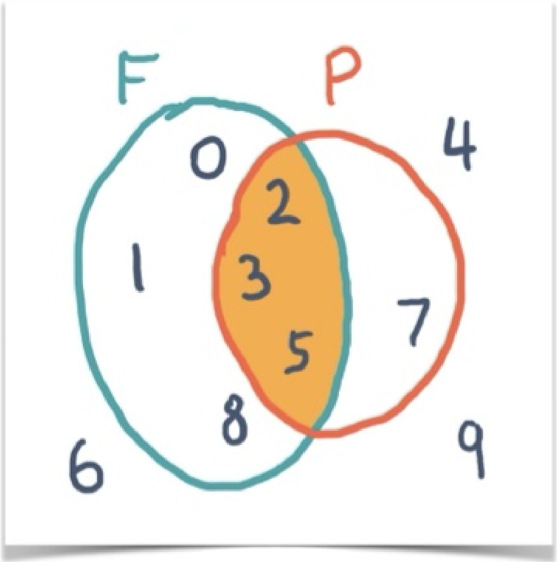
is the same as:

x belongs to A ***and***
x also belongs to B.

Math notation:

$$x \in (A \cap B) \iff (x \in A) \land (x \in B)$$

In computing: **AND**



LOGIC DISJUNCTION: UNION

x belongs to the union of A and B.

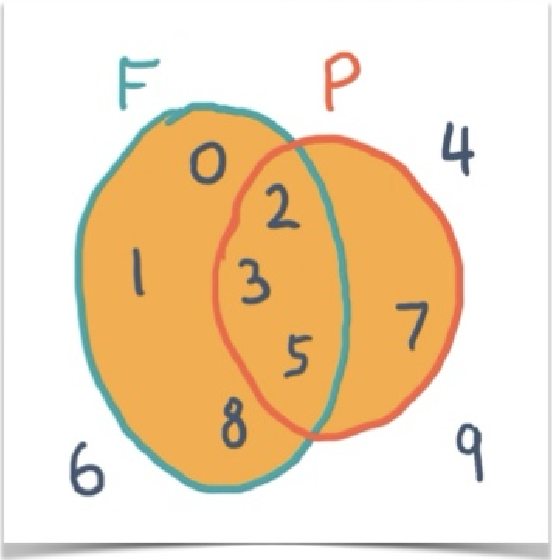
is the same as:

x belongs to A ***or***
x belongs to B.

Math notation:

$$x \in (A \cup B) \Longleftrightarrow (x \in A) \vee (x \in B)$$

In computing: **OR**

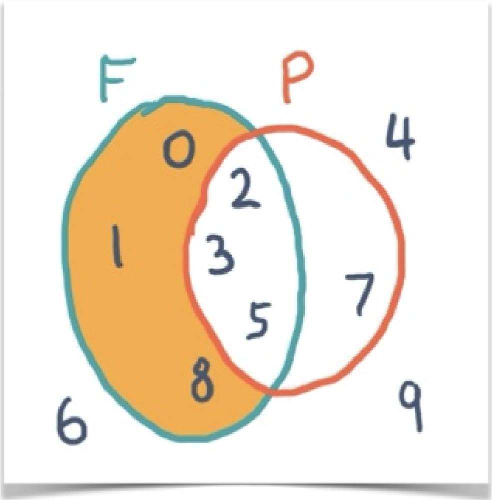


DIFFERENCE

x belongs to A but does not belong to B.

is the same as:

elements of A minus elements of B



Math notation:

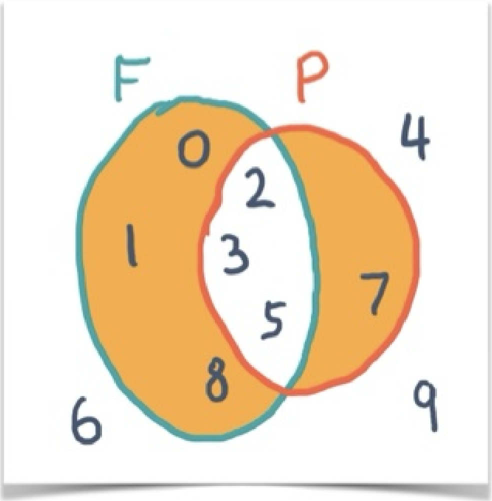
$$x \in (A \setminus B) \Longleftrightarrow (x \in A) \land (x \notin B)$$

SYMMETRIC DIFFERENCE

x belongs to A ***or***
x belongs to B but
does not belong to both

Is the same as:

x belongs to the union of A with B
less the intersection of A with B.



Math notation:

$$x \in (A \,\Delta\, B) \iff (x \in A) \veebar (x \in B)$$

In computing: **X0R**

Set algebra in the wild

Set algebra

- Essential: $e \in S$
- Highly desirable in practice:
  - $S \supseteq Z$ (superset: S contains all elements of Z)
  - $S \cap Z$ (intersection)
  - $S \cup Z$ (union)
  - $S \setminus Z$ (difference)

Implementing sets

- Invariant: set elements are unique
  - Elements must be hashable and comparable
- $e \in S$ (membership test):
  - Expected to be $O(1)$ in a reasonable implementation
- Membership test at constant time provides significant performance gains with the remaining operations

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	**MapSet:** 14 methods	😻
Ruby	**Set**: > 10 methods and operators	😻
Python	**set**, **frozenset**: > 10 methods and operators	😻
.Net (C# etc.)	**ISet** interface: > 10 methods; 2 implementations	😻
JavaScript (ES6)	**Set**: < 10 methods	
Java	**Set** interface: < 10 methods; 8 implementations	
Go	Do it yourself, or pick one of several packages...	

vintage
2018

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	**MapSet:** 14 methods	😻
Ruby	**Set**: > 10 methods and operators	😻
Python	**set**, **frozenset**: > 10 methods and operators	😻
.Net (C# etc.)	**ISet** interface: > 10 methods; 2 implementations	😻
JavaScript (ES6)	**Set**: < 10 methods	😿
Java	**Set** interface: < 10 methods; 8 implementations	😿
Go	Do it yourself, or pick one of several packages...	😾

vintage
2018

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	**MapSet:** 14 methods	😻
Ruby	**Set**: > 10 methods and operators	😻
Python	**set**, **frozenset**: > 10 methods and operators	😻
.Net (C# etc.)	**ISet** interface: > 10 methods; 2 implementations	😻
JavaScript (ES6)	**Set**: < 10 methods	😿
Java	**Set** interface: < 10 methods; 8 implementations	😿
Go	Do it yourself, or pick one of several packages…	😾

rich APIs

lean APIs

vintage 2018

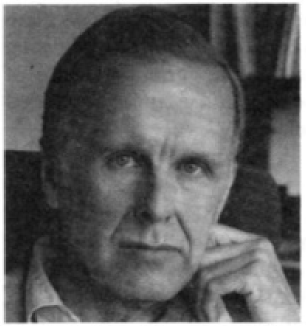
Evolution of set APIs

- Most methods in lean APIs handle single elements
  - e.g.: adding/removing an element, iteration
- “The von Neumann bottleneck”

JOHN BACKUS — TURING AWARD LECTURE, 1977

Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs

John Backus
IBM Research Laboratory, San Jose



Author's address: 91 Saint Germain Ave., San Francisco, CA 94114.

Conventional programming languages are growing ever more enormous, but not stronger. Inherent defects at the most basic level cause them to be both fat and weak: their primitive word-at-a-time style of programming inherited from their common ancestor—the von Neumann computer, their close coupling of semantics to state transitions, their division of programming into a world of expressions and a world of statements, their inability to effectively use powerful combining forms for building new programs from existing ones, and their lack of useful mathematical properties for reasoning about programs.

An alternative functional style of programming is founded on the use of combining forms for creating programs. Functional programs deal with structured data, are often nonrepetitive and nonrecursive, are hierarchically constructed, do not name their arguments, and do not require the complex machinery of procedure declarations to become generally applicable. Combining forms can use high level programs to build still higher level ones in a style not possible in conventional languages.

Evolution of set APIs

- Languages are providing more set algebra operations
- ECMAScript 2025 added:

```
Set.intersection, Set.union,
Set.difference,
Set.symmetricDifference,
Set.isSubsetOf, Set.isSupersetOf,
Set.isDisjointFrom
```

- Go 1.28 will have a rich set API!

The collections proposal for Go 1.28

The Go Collections working group was formed in late 2025 with the purpose of bringing common collection data structures to the standard library, guided by the familiar Go principles of pragmatism and simplicity.

…

This work seeks to add several of the more important data types to the standard library, and to establish conventions for their APIs and those of future additions.

Alan Donovan
Go issue #80590

The 7 proposed additions

1) `hash/maphash.Hasher`
   standard interface supporting custom hash functions and equivalence relations for arbitrary data types (released in Go 1.27)
2) `container/hash.Map[K,V]`
   a hash-based Map that uses custom hash functions.
3) `container/hash.Set[T]`
   a hash-based Set along the same lines.
4) `container/heap/v2.Heap`
   a generic binary heap API to replace the standard library's existing heap (hard to use).

The 7 proposed additions

5) `container/set.Set[T]`
   transparently represented as map[T]struct{}, supporting usual set operations such as Union and Intersection. We expect it to become the standard set in most new Go APIs.

6) `container/mapset`
   helper functions (Union, Intersection, and so on) for manipulating legacy sets as sets in existing code whose API cannot be changed. set.Set wraps this API.

7) `container/ordered.Map[K,V]`
   a map that preserves insertion order

proposal: container/...: generic collection types

- Umbrella issue: https://github.com/golang/go/issues/80590
- 6 new concrete collection types
- 1 new interface `Hasher` (included in Go 1.27)
- 3 new abstract interfaces for collections, sets and maps

Source tree (WIP)

Open Cls (unmerged) identified by #issue

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go        =  Hasher interface, in Go 1.27 (#70471)
├── maps/
│   └── maps.go               ★  added Identical (#78456), used by mapset
├── container/
│   ├── container_test.go     ★+ abstract interfaces (unexported)
│   ├── set/
│   │   └── set.go            ★+ the new canonical set (#69230)
│   ├── mapset/
│   │   └── mapset.go         ★+ helpers for map-based sets (#77052)
│   ├── hash/
│   │   ├── map.go            ★+ hash.Map, uses maphash.Hasher (#69559)
│   │   └── set.go            ★+ hash.Set, same approach (#80584)
│   ├── heap/
│   │   ├── heap.go           =  existing, pre-generics
│   │   └── v2/
│   │       └── heap.go       ?  generic, more ergonomic Heap (#77397)
│   ├── list/
│   │   └── list.go           =  existing
│   ├── ordered/
│   │   └── ordered.go        ?  tree-based ordered.Map/Set (#60630)
│   └── ring/
│       └── ring.go           =  existing
│
```

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   ├── hash/
│   │   ├── map.go
│   │   └── set.go
```

Abstract interfaces

Unexported but
intended as models
for new collection types

Comment in `container/container_test.go`

```
// The following interfaces define the abstract data types for
// collections in Go. They are expressed using F-bounded polymorphic
// interfaces to achieve covariant parameter/result specialization,
// and may be used as constraint types in generic functions.
//
// These interfaces are not yet published, but may be included in a
// future Go release once we have experience of whether these methods
// are necessary and sufficient.
```

_AbstractSet

```go
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
    _AbstractCollection[E, S]

    Insert(E) bool
    InsertAll(iter.Seq[E]) bool
    Equal(S) bool
    All() iter.Seq[E]
    Delete(E) bool
    DeleteAll(iter.Seq[E]) bool
    DeleteFunc(func(E) bool) bool
    Intersection(S) S
    IntersectionWith(S)
    Intersects(S) bool
    Union(S) S
    UnionWith(S)
    Difference(S) S
    DifferenceWith(S)
    SymmetricDifference(S) S
    SymmetricDifferenceWith(S)
}
```

```go
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
    _AbstractCollection[E, S]

    Insert(E) bool
    InsertAll(iter.Seq[E]) bool
    Equal(S) bool
    All() iter.Seq[E]
    Delete(E) bool
    DeleteAll(iter.Seq[E]) bool
    DeleteFunc(func(E) bool) bool
    Intersection(S) S
    IntersectionWith(S)
    Intersects(S) bool
    Union(S) S
    UnionWith(S)
    Difference(S) S
    DifferenceWith(S)
    SymmetricDifference(S) S
    SymmetricDifferenceWith(S)
}
```

return bool to report change

```go
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
    _AbstractCollection[E, S]

    Insert(E) bool
    InsertAll(iter.Seq[E]) bool
    Equal(S) bool
    All() iter.Seq[E]
    Delete(E) bool
    DeleteAll(iter.Seq[E]) bool
    DeleteFunc(func(E) bool) bool
    Intersection(S) S  👈
    IntersectionWith(S)
    Intersects(S) bool
    Union(S) S  👈
    UnionWith(S)
    Difference(S) S  👈
    DifferenceWith(S)
    SymmetricDifference(S) S  👈
    SymmetricDifferenceWith(S)
}
```

set algebra operations return new set

```go
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
	_AbstractCollection[E, S]

	Insert(E) bool
	InsertAll(iter.Seq[E]) bool
	Equal(S) bool
	All() iter.Seq[E]
	Delete(E) bool
	DeleteAll(iter.Seq[E]) bool
	DeleteFunc(func(E) bool) bool
	Intersection(S) S
	IntersectionWith(S)  👈
	Intersects(S) bool
	Union(S) S
	UnionWith(S)
	Difference(S) S
	DifferenceWith(S)  👈
	SymmetricDifference(S) S
	SymmetricDifferenceWith(S)  👈
}
```

update receiver in-place

_AbstractSet: a self-referential declaration!

```
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
```

`Enum` is actually a generic class defined as `Enum<T extends Enum<T>>`. This circular definition is probably the most confounding generic type definition you are likely to encounter. We're assured by the type theorists that this is quite valid and significant, and that we should simply not think about it too much, for which we are grateful.

Ken Arnold, James Gosling, David Holmes

The Java Programming Language, 4th Edition

Cited in Generics Considered Harmful

by Ken Arnold (https://fpy.li/15-51)

A good, informal explanation

- F-Bounded Polymorphism: Type-Safe Builders in Java
  - How a self-referential type bound solves a real inheritance problem, why Java's own Enum uses the same trick, and what it all means in practice.
  - By Pradeep Samuel
  - Published March 9, 2026

https://www.fbounded.com/blog/f-bounded-polymorphism/

_AbstractSet is F-bounded

```
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
```

- That's Go notation for an
  "F-bounded polymorphic interface"
- Provides de S type variable,
  useful in return types for the set algebra
- Constrains the concrete type of S to
  a subtype of `_AbstractSet`

```go
// _AbstractSet models a set S of elements E,
// such as *hash.Set, or set.Set.
type _AbstractSet[E any, S _AbstractSet[E, S]] interface {
	_AbstractCollection[E, S]

	Insert(E) bool
	InsertAll(iter.Seq[E]) bool
	Equal(S) bool
	All() iter.Seq[E]
	Delete(E) bool
	DeleteAll(iter.Seq[E]) bool
	DeleteFunc(func(E) bool) bool
	Intersection(S) S  👈
	IntersectionWith(S)
	Intersects(S) bool
	Union(S) S  👈
	UnionWith(S)
	Difference(S) S  👈
	DifferenceWith(S)  👈
	SymmetricDifference(S) S  👈
	SymmetricDifferenceWith(S)  👈
}
```

_AbstractMap is also F-bounded

```go
// _AbstractMap models a mapping M from keys K to values V,
// such as *hash.Map or *ordered.Map.
type _AbstractMap[K, V any, M _AbstractMap[K, V, M]] interface {
    _AbstractCollection[K, M]

    Set(K, V) (V, bool)
    SetAll(iter.Seq2[K, V]) bool
    Get(K) (V, bool)
    At(K) V
    All() iter.Seq2[K, V]
    Keys() iter.Seq[K]
    Values() iter.Seq[V]
    Delete(K) (V, bool)
    DeleteAll(iter.Seq[K]) bool
    DeleteFunc(func(K, V) bool) bool
}
```

F-bounded _AbstractCollection

```go
// _AbstractCollection models a collection C of elements E,
// such as *hash.Map, *hash.Set, *ordered.Map, or set.Set.
type _AbstractCollection[E any, C _AbstractCollection[E, C]] interface {
    Clear()
    Clone() C
    Contains(E) bool
    ContainsAll(iter.Seq[E]) bool
    Len() int
    String() string
}
```

Both _AbstractSet and _AbstractMap embed _AbstractCollection

After the interfaces in `container_test.go`

```
// These types define the fundamental operations that need to be
// implemented by all set and map types. Their naming, signature, and
// semantic conventions should be followed wherever possible when
// defining new collection types.
// ...
//
// Map.Set should replace an existing entry with an equivalent key.
// This follows the built-in map: https://go.dev/play/p/pkH8kkFTuEg.
//
// The "plural" functions {Contains,Delete,Insert,Set}All are
// sufficiently important that they belong as methods; they
// also compute a convenient bool result.
```

Static tests in `container_test.go`

```go
// -- conformance --

// This is a static compilation test of various symmetries,
// expressed using F-bounded polymorphic interfaces to
// achieve covariant parameter/result specialization.

var _ _AbstractSet[int, set.Set[int]] = make(set.Set[int])

var _ _AbstractSet[int, *hash.Set[int]] = new(hash.Set[int])
```

The compiler allows assigning `set.Set` and `hash.Set` to a variable `_` of type `_AbstractSet`

Generic function in `container_test.go`

```go
// ContainsAny reports whether set x contains any element of sequence y
func ContainsAny[E any, S _AbstractSet[E, S]](x S, y iter.Seq[E]) bool {
	for elem := range y {
		if x.Contains(elem) {
			return true
		}
	}
	return false
}
```

`ContainsAny` works with any `_AbstractSet`

Another generic function in `container_test.go`

```go
// Take removes and returns an arbitrary element from a set.
// It returns zero if the set was empty.
func Take[S _AbstractSet[E, S], E any](set S) (e E, found bool) {
	for e = range set.All() {
		found = true
		set.Delete(e) // may fail for NaN
		break
	}
	return
}
```

Take returns an element of type E (from `_AbstractSet[E, S]`) and a `bool`

More set algebra, also in `container_test.go`

```go
// Subset reports whether set x is a subset of set y.
func Subset[S _AbstractSet[E, S], E any](x, y S) bool {
	// We cannot shortcut if x == y here because
	// it may panic for some types (e.g. maps) or give
	// the wrong answer for others (e.g. strings
	// considered as unordered sets of bytes).
	// Secondarily, pointer identity also doesn't
	// repect NaN != NaN.

	return x.Len() <= y.Len() && y.ContainsAll(x.All())
}

// Superset reports whether x is a superset of y.
func Superset[S _AbstractSet[E, S], E any](x, y S) bool {
	return Subset(y, x)
}
```

$x \subseteq y$

$x \supseteq y$

Implementation of `set.Set[T]`

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   ├── hash/
│   │   ├── map.go
│   │   └── set.go
```

set.Set[T]

`set.Set` delegates all work to `mapset` functions

```go
package set

import (
  "github.com/ramalho/sets-in-go/vendored/mapset"  // proposed mapset
  "iter"
  "maps"
)

// A Set[E] is a set of elements of type E.
type Set[E comparable] map[E]struct{}

// Collect creates a new set containing the elements of the sequence.
func Collect[E comparable](seq iter.Seq[E]) Set[E] {
  return Set[E](mapset.Collect(seq))
}
```

All but one of the functions and methods in `set.go` are one-liners calling functions in `mapset`

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   ├── hash/
│   │   ├── map.go
│   │   └── set.go
```

Functions for using legacy maps as sets

The underlying mapset (uses no named types)

```go
package mapset

import (
	"iter"
)

// Collect returns a new set containing the elements of the sequence.
func Collect[K comparable](seq iter.Seq[K]) map[K]struct{} {
	return collect[K, struct{}](seq)
}

/// ...

func collect[K comparable, V bool | struct{}](seq iter.Seq[K]) map[K]V {
	x := make(map[K]V)
	InsertAll(x, seq)
	return x
}
```

mapset.InsertAll and mapset.insert

```go
// InsertAll adds each element of the addenda sequence to the set.
// If the set values are boolean, the value 'true' is used.
// It reports whether len(x) changed.
func InsertAll[M ~map[K]V, K comparable, V bool | struct{}](x M, addenda iter.Seq[K]) bool {
	pre := len(x)
	for k := range addenda {
		insert(x, k)
	}
	return len(x) != pre
}

func insert[M ~map[K]V, K comparable, V bool | struct{}](m M, k K) {
	// Choose the distinguished "present" value (true or struct{}{}).
	// This compiles to a load from .rodata.
	var present V
	if _, ok := any(present).(bool); ok {
		present = any(true).(V)
	}

	// This is the canonical insertion operation.
	// All maps created by this API use only the
	// distinguished 'present' value for the result type.
	m[k] = present
}
```

Hasher interface

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   ├── hash/
│   │   ├── map.go
│   │   └── set.go
```

Set type with support for custom hashers

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   ├── hash/
│   │   ├── map.go
│   │   └── set.go
```

Wrapping up

Key takeaways

- Set algebra provides simpler, faster solutions to common data processing tasks.
- Sets and other collections in the upcoming Go 1.28 provide a foundation for the design of generic collections.
- The Go source code shown here is unmerged and may change!

Mandatory AI tips

- For developers using agents:
  The precise vocabulary of set algebra is useful to prompt coding agents, regardless of the programming language.
- For citizen programmers:
  Learning set algebra and the relational model may be the best foundation for agentic coding.
- For everyone:
  Demand your time back. Time is not money, it is your time to live!

Remember Aaron Schwartz

- Co-creator of RSS and Markdown
- Co-founder of Reddit
- Aaron was arrested for downloading scientific articles from JSTOR in bulk
- His life was destroyed by the Obama DOJ
- He never shared the articles with anyone.
- What would he would do with them? Perhaps train a model?

The Knowledge Smoothie

SAM ALTMAN: Let’s make a smoothie! Who can contribute with fruits?

EVERYONE: OK, here’s the fruit I have!

SAM ALTMAN: The smoothie is ready. It’s $20 a glass!

EVERYONE: But you took our fruits to make it!

SAM ALTMAN: No I didn’t! Where’s your fruit? Show me!

The Solution

- Any LLM vendor who refuses to disclose every external source used in training must open source the model and its weights.
- Too radical?
- It’s only fair and common sense.